Basics

20 May 2021 10:52

REST (Representational State Transfer) is an architectural style for designing networked applications. RESTful APIs adhere to the principles of REST architecture, providing a standardized way for systems to communicate over the internet.

Resources: In REST, everything is considered a resource, which can be accessed using a unique identifier, typically a URL (Uniform Resource Locator).

HTTP Methods: RESTful APIs use standard HTTP methods such as GET, POST, PUT, and DELETE to perform CRUD (Create, Read, Update, Delete) operations on resources.

Representations: Resources are represented in a format such as JSON (JavaScript Object Notation) or XML (eXtensible Markup Language), allowing for easy exchange of data between client and server.

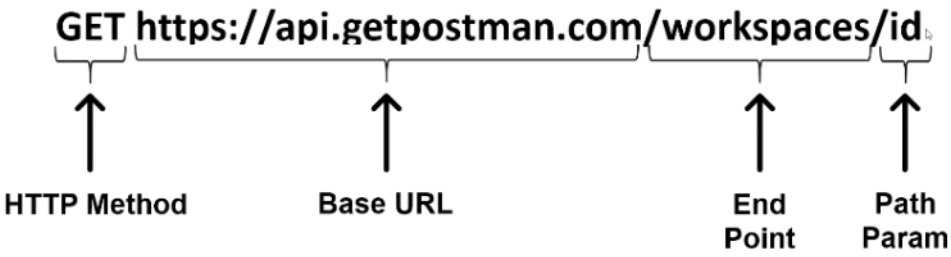
Constraint	Description	Implication
Statelessness	Each request must contain all the information needed by the server.	Simplifies server design and scalability; no session state is stored between requests.
Client-Server Architecture	Separation between client and server.	Allows independent evolution of client and server; defines clear responsibilities.
Uniform Interface	Standardized way of interacting with resources through fixed methods and URIs.	Simplifies architecture, improves usability, and ensures consistency.
Resource-Based	API is designed around resources identified by URIs.	Resources are accessed and manipulated using standard HTTP methods, improving clarity and organization.
Layered System	Architecture can be composed of multiple layers, each with specific functions.	Enhances scalability and flexibility; changes can be made independently at different layers.
Cacheability	Responses should indicate if they are cacheable.	Improves performance by reducing redundant requests; proper caching is crucial for efficiency.

RESTful APIs provide a powerful and flexible way to build networked applications, enabling interoperability, scalability, and performance.

HTTP is the underlying protocol used by RESTful APIs, and REST builds on top of HTTP to provide a set of principles for designing scalable and interoperable web services. Rest can also use FTP or SMTP.

Safe: Doesn't change resources on server
Idempotent: performing the same operation multiple times produces the same result as performing it once

Method	Description	Request Body	Response Body	Safe	Idempotent	Cacheable
GET	Transfer a current representation of the target resource	No	Yes	Yes	Yes	Yes
HEAD	Same as GET, but only transfer the status line and header section	No	No	Yes	Yes	Yes
POST	Perform resource-specific processing on the request payload	Yes	Yes	No	No	In some cases
PUT	Replace all current representations of the target resource with the request payload	Yes	No	No	Yes	No
DELETE	Remove all current representations of the target resource	Optional	Optional	No	Yes	No
CONNECT	Establish a tunnel to the server identified by the target resource	No	Yes	No	No	No
OPTIONS	Describe the communication options for the target resource	No	Yes	Yes	Yes	No
TRACE	Perform a message loop-back test along the path to the target resource	No	No	Yes	Yes	No
PATCH	Perform partial modification of the target resource	Yes	Yes	No	No	No



Endpoint: The address where the API is hosted on the server.

Path	Variable parts of a URL path used to point to a specific resource within a collection, like a user identified by ID.

Parameters	- https://www.google.com/Images/1123343 - https://www.google.com/docs/1123343 - https://amazon.com/orders/112 given().baseUri("https://reqres.in").pathParam("userId","2").log().all().when().get("/api/users/{userId}").then().log().all().assertThat().statusCode(200);

Query Parameters	Used to sort/filter resources, identified with "?"
	Query parameters are separated by an ampersand (&) if there are multiple parameters.
	
	- https://amazon.com/orders?sort_by=2/20/2020
	
	HashMap<String, String> queryParams = new HashMap<>();
	queryParams.put("foo1", "bar1");
	queryParams.put("foo2", "bar2");
	
	given().baseUri("https://postman-echo.com").param("foo1", "bar1").queryParam("foo1", "bar1").queryParam("foo2", "bar2").queryParams(queryParams).log().all().when().get("/get").then().log().all().assertThat().statusCode(200);

Form Parameters	Attachments are sent as form parameters and are safe from security perspective • Sending Parameters: given().formParam("key", "value") • Sending Files: given().multiPart("file", new File("path/to/file"))

Category	Status Code	
Informational	100-199:Indicates the request has been received and is being processed.	
Successful	200-299	Indicates that the request was successfully received, understood, and accepted.
	200	OK: The request was successful.
	201	Created: The resource was successfully created.
	204	No Content: The request was processed, but there is no content to send back.
Redirection	300-399	Indicates that further action is needed to complete the request.
	301	Moved Permanently: The resource has been moved to a new location.
Client Error	400-499	Indicates there was an error with the client's request.
	400	Bad Request: The request was malformed or missing necessary information.
	401	Unauthorized: Authentication failed (e.g., invalid or expired token).
	403	Forbidden: The client does not have permission to access the resource.
	404	Not Found: The requested resource could not be found.
Server Error	500-599	Indicates the server failed to fulfill a valid request.
	500	Internal Server Error: A generic error indicating something went wrong on the server.
	502	Bad Gateway: Received an invalid response from an upstream server.
	503	Service Unavailable: The server is temporarily unable to handle the request.
	504	Gateway Timeout: The gateway server did not receive a timely response.

Hamcrest

16 March 2024 10:08

Hamcrest is a framework for writing matcher objects in Java. It provides a fluent interface for creating assertions in a morereadable and expressive way.

- equalTo: Checks if the actual value is equal to the expected value.
- containsString: Checks if the actual value contains the expected substring.
- hasItems: Verifies that the actual list or array contains the specified items.
- not: Used to negate other matchers.
- greaterThan, lessThan, greaterThanOrEqualTo, lessThanOrEqualTo: Used to compare numbers.

```
body("userId", equalTo(1)).
body("id", greaterThan(0)).
body("title", containsString("sunt")).
body("body", notNullValue());
```

Anonymous JSON root validation: [1, 2, 3]:body("$", hasItems(1, 2, 3)); // An empty string "" would work as well

Passing Headers

16 March 2024 12:32

Method	Example
Single Header (using header() method)	header("header", "value1")
Multiple Headers (using header() method)	Header header = new Header("header", "value1"); Header matchHeader = new Header("x-mock-match-request-headers", "header"); .header(header).header(matchHeader)
Multiple Headers (using Map)	HashMap<String, String> headers = new HashMap<>(); headers.put("header", "value2"); .headers(headers)

Multiple Headers as String Parameters	Headers extractedHeaders = given().baseUri("https://8f6d7436-aba9-4c1f-bc81-fdc881a11fb1.mock.pstmn.io").headers(headers).when().get("/get").then().assertThat().statusCode(200).extract().headers();
	for (Header header : extractedHeaders) {
	System.out.print("header name = " + header.getName() + ", ");
	System.out.println("header value = " + header.getValue());
	}

Assert Response Headers	.headers("responseHeader", "resValue1", "X-RateLimit-Limit", "120")
Extract Response Headers	.extract().headers()
Extract Multi-Value Header	List<String> values = extractedHeaders.getValues("multiValueHeader")

Logging

15 August 2024 12:50

- log().all()
- log().headers()
- log().body()
- log().cookies()
- log().parameters()
- log().status()
- log().statusCode()
- log().everything()
- log().ifError()
- log().ifValidationFails()

Logging to a File

```
 @BeforeClass
   public void beforeClass() throws FileNotFoundException {
      PrintStream fileOutPutStream = new PrintStream("restAssured.log");

      RequestSpecBuilder requestSpecBuilder = new RequestSpecBuilder().addFilter(new RequestLoggingFilter(fileOutPutStream)).addFilter(new
ResponseLoggingFilter(fileOutPutStream));

      requestSpecification = requestSpecBuilder.build();

      ResponseSpecBuilder responseSpecBuilder = new ResponseSpecBuilder();
      responseSpecification = responseSpecBuilder.build();
   }

   @Test
   public void loggingFilter() {
      given(requestSpecification).baseUri("https://postman-
echo.com").log().all().when().get("/get").then().spec(responseSpecification).log().all().assertThat().statusCode(200);
   }
```

Request/Response Specification/Builder

28 June 2021 10:04

Request Specification

- Purpose: An interface that can be used to group and retrieve repetitive actions, such as setting up the base URL, headers, and HTTP verbs.
- Benefits: Reduces the number of lines of code and increases maintainability by reusing common configurations.
- Usage: Define common request configurations that can be applied to multiple requests.

Response Specification

- Purpose: An interface that allows you to specify how the expected response must look like for a test to pass.
- Benefits: Used to validate a common response or a response needed for multiple tests, ensuring consistency in response validation.

RequestSpecBuilder

- Purpose: Builder class for constructing `RequestSpecification` instances.
- Used for: Creating and configuring complex or reusable `RequestSpecification` instances.
- Method: Use `RequestSpecBuilder` to set up parameters and call `build()` to get a `RequestSpecification`.
- Example:

```
 RequestSpecification requestSpec = new RequestSpecBuilder()
    .setBaseUri("https://rahulshettyacademy.com")
    .addQueryParam("key", "qaclick123")
    .setContentType(ContentType.JSON)
    .build();
```

ResponseSpecBuilder

- Purpose: Builder class for constructing `ResponseSpecification` instances.
- Used for: Creating and configuring complex or reusable `ResponseSpecification` instances.
- Method: Use `ResponseSpecBuilder` to set up expected responses and call `build()` to get a `ResponseSpecification`.
- Example:

```
 ResponseSpecification responseSpec = new ResponseSpecBuilder()
    .expectStatusCode(200)
    .expectHeader("server", "Apache/2.4.52 (Ubuntu)")
    .build();
```

Example Usage

```
RestAssured.baseURI = "https://rahulshettyacademy.com";

   // Define request specification
   RequestSpecification requestSpec = new RequestSpecBuilder()
         .setBaseUri(RestAssured.baseURI)
         .addQueryParam("key", "qaclick123")
         .setContentType(ContentType.JSON)
         .build();

   // Define response specification
   ResponseSpecification responseSpec = new ResponseSpecBuilder()
         .expectStatusCode(200)
         .expectHeader("server", "Apache/2.4.52 (Ubuntu)")
         .build();

   // Perform POST request
   String postResponse = given().spec(requestSpec)
         .body(new File("src/test/resources/AddPlace.json"))
         .when().post("maps/api/place/add/json")
         .then().spec(responseSpec)
         .body("scope", equalTo("APP"))
         .extract().response().asString();
```

Serialization

28 June 2021 09:57

Serialization is a process where you convert an Instance of a Class (Object of a class) into a Byte Stream

```
String filePath = "src/test/resources/complex.json";
String jsonString = new String(Files.readAllBytes(Paths.get(filePath)));
```

Library	Dependency	Usage

Jackson	<dependency>
	<groupId>com.fasterxml.jackson.core</groupId>
	<artifactId>jackson-databind</artifactId>
	<version>2.13.1</version> </dependency>
		@Test
		public void orgJacksonMockSerialization() throws JsonProcessingException {
		// Parse JSON String
		ObjectMapper objectMapper = new ObjectMapper();
		Object jsonObject = objectMapper.readValue(jsonString, Object.class);
		
		// Serialize JSON Object_Pretty
		String serializedJson =
		objectMapper.writerWithDefaultPrettyPrinter().writeValueAsString(jsonObject);
		System.out.println(serializedJson);
		}
		
		@Test
		public void orgJacksonPOJOSerialization() throws JsonProcessingException {
		ObjectMapper objectMapper = new ObjectMapper();
		String serializedJson = objectMapper.writeValueAsString(setAddPlace());
		System.out.println(serializedJson);
		}

org.json	<dependency>
	<groupId>org.json</groupId>
	<artifactId>json</artifactId>
	<version>20210307</version>
	</dependency>
		@Test
		public void orgJsonMockSerialization() {
		JSONObject jsonObject = new JSONObject(jsonString);
		String serializedJson = jsonObject.toString(4); // Indented with 4 spaces
		System.out.println(serializedJson);
		}
		
		@Test
		public void orgJsonPOJOSerialization(){
		JSONObject jsonObject = new JSONObject(setAddPlace());
		System.out.println(jsonObject.toString(4));
		}

Gson	<dependency>
	<groupId>com.google.code.gson</groupId>
	<artifactId>gson</artifactId>
	<version>2.8.7</version>
	</dependency>
		@Test
		public void orgGsonPOJOSerialization(){
		Gson gson = new Gson();
		String serializedJson = gson.toJson(setAddPlace());
		System.out.println(serializedJson);
		// Pretty-print JSON
		Gson gsonPretty = new GsonBuilder().setPrettyPrinting().create();
		String prettyJson = gsonPretty.toJson(setAddPlace());
		System.out.println(prettyJson);
		}
		
		@Test
		public void orgGsonMockSerialization() {
		Gson gson = new Gson();
		Object jsonObject = gson.fromJson(jsonString, Object.class);
		String serializedJson = gson.toJson(jsonObject);
		System.out.println(serializedJson);
		}

Derialization

28 June 2021 09:58

Serialization in Rest Assured context is a process of converting a Java object into Request body (Payload). Rest Assured also Supports deserialization by converting Response body back to Java object.

Advantages:

- Easy to parse and extract response (Json/XML) values if they are wrapped as Java object.
- User-friendly Methods can be created which makes code more readable.

Design Approach:

- Java object is constructed with the support of POJO classes.
- POJO classes are created based on the request/Response payload.

Additional Libraries: For JSON, you need to have either Jackson, Jackson2, Gson, or Johnzon in the classpath. For XML, you need JAXB.

```
static class Book {
    public String name;
    public String isbn;
    public String aisle;
    public String author;
  }
```

Jackson	@Test
	public void JacksonDeserializationExample() throws JsonProcessingException {
	String jsonResponse = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author\": \"John foer\" }";
	
	// Deserialize JSON response using Jackson
	ObjectMapper objectMapper = new ObjectMapper();
	Book book = objectMapper.readValue(jsonResponse, Book.class);
	
	System.out.println("Book Name: " + book.name);
	System.out.println("Author: " + book.author);
	}

GSON	@Test
	public void GsonDeserializationExample (){
	String jsonResponse = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author\": \"John foer\" }";
	
	// Deserialize JSON response using Gson
	Gson gson = new Gson();
	Book book = gson.fromJson(jsonResponse, Book.class);
	
	System.out.println("Book Name: " + book.name);
	System.out.println("Author: " + book.author);
	}

Org.json	@Test
	public void OrgJsonDeserializationExample (){
	String jsonResponse = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author\": \"John foer\" }";
	
	// Deserialize JSON response using org.json
	JSONObject jsonObject = new JSONObject(jsonResponse);
	
	String name = jsonObject.getString("name");
	String author = jsonObject.getString("author");
	
	System.out.println("Book Name: " + name);
	System.out.println("Author: " + author);
	}

JSONPAth	@Test public void JsonPathDeserializationExample(){ String jsonResponse = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author\": \"John foer\" }";

```
    // Parse JSON using JsonPath
    JsonPath jsonPath = new JsonPath(jsonResponse);

    // Extract specific values
    String name = jsonPath.getString("name");
    String author = jsonPath.getString("author");

    System.out.println("Book Name: " + name);
    System.out.println("Author: " + author);
}
```

Authentication

20 July 2021 09:46

Authentication Method	Description	Details

Basic Authentication(Not Secure)	Basic access authentication is a method for an HTTP user agent (e.g., a web browser) to provide a username and password when making a request.
	
	Response response = RestAssured
	.given()
	.auth()
	.basic(username, password)
	.when()
	.get(baseUrl);
	
	Response response = RestAssured.given()
	.auth().preemptive().basic("username", "password") // Preemptive Basic authentication
	.when()
	.get("https://example.com/protected-resource");
		Basic authentication sends the username and password in the request headers. The credentials are sent in an easily decodable format (Base64 encoded), which is less secure unless the communication is encrypted (e.g., using HTTPS).
		
		Basic Authentication is normally done after receiving a 401 Unauthorized response, whereas Preemptive Authentication sends the credentials right away without waiting for the 401 status

Digest Authentication	Digest authentication is a method used by web servers to negotiate credentials securely. It involves hashing the username and password along with other parameters, making it more secure than plain text transmission. Response response = RestAssured .given() .auth() .digest(username, password) .when() .get(baseUrl);	Process: - HA1 = MD5(username:realm) - HA2 = MD5(method) - Response = MD5(HA1:nonce) Digest authentication is a more secure alternative to basic authentication. Instead of sending the username and password directly, the client sends a hashed value of the credentials and other information (like a random challenge) that is calculated on both the client and server sides.
Bearer Authentication	Bearer authentication is a token-based authentication mechanism where a bearer token is passed in the HTTP header to authorize requests. This token is usually obtained through OAuth or another authentication mechanism. Response response = RestAssured.given() .header("Authorization", "Bearer " + bearerToken) .when() .get("https://example.com/protected-resource");	Process: - Obtain Bearer Token from an authentication server. - Include Authorization: Bearer <token> in the HTTP request header. - Token must be valid and not expired.

API Key Authentication	Response response = RestAssured
	.given()
	.header("X-API-Key", apiKey) // Add API key to the request header
	.when()
	.get(baseUrl);	

Form Authentication	SessionFilter filter = new SessionFilter();
	
	Response loginResponse = RestAssured.given()
	.formParam("username", "your-username") // Username parameter
	.formParam("password", "your-password") // Password parameter
	.when()
	.post(loginUrl);
	
	Response protectedResourceResponse = RestAssured.given()
	.cookies(loginResponse.getCookies()) // Pass the session cookies from the login response
	.when()
	.get(protectedResourceUrl);
		Form-based authentication is a method where the user submits a form (often including a username and password) to authenticate against a web application. In this type of authentication, credentials are typically sent as form parameters (such as username and password), and the server issues a session or token for subsequent requests.

OAuth 2.0(Authorization Only)	OAuth 2.0 is a more advanced and flexible version of OAuth that provides a framework for token-based authorization. It supports various grant types, including authorization code, client credentials, and refresh tokens. Example using google photos to share your photos with 3rd party apps. // 1. First, obtain the access token String clientId = "your-client-id"; String clientSecret = "your-client-secret";	Grant Types: - Authorization Code: Used for web and mobile applications. - Client Credentials: Used for server-to-server communication. - Password Credentials: Used for user-to-server communication. - Refresh Token: Used to obtain new access tokens.

	String tokenUrl = "https://oauth-server.com/token"; // Token endpoint URL
	
	Response tokenResponse = RestAssured.given()
	.auth().preemptive().basic(clientId, clientSecret) // Basic authentication for client credentials
	.contentType(ContentType.URLENC) // Content type for OAuth token request
	.formParam("grant_type", "client_credentials") // Grant type for OAuth
	.when()
	.post(tokenUrl);
	
	// Extract access token from the response
	String accessToken = tokenResponse.jsonPath().getString("access_token");
	
	// 2. Now, use the access token to authenticate requests to the API
	Response apiResponse = RestAssured.given()
	.auth().oauth2(accessToken) // OAuth2 authentication using access token
	.when()
	.get("https://api.example.com/protected-resource");
		Usage: Include Authorization: Bearer <token> in the HTTP request header.

Authorization Code Flow(Social Login)	OpenID Connect (OIDC) is an identity layer built on top of the OAuth 2.0 protocol. It allows for authentication and provides a way to verify the identity of users based on the authentication performed by an authorization server. It also allows for the retrieval of basic profile information about the user. String tokenEndpoint = "https://authorization-server.com/oauth2/token"; String clientId = "yourClientId"; String clientSecret = "yourClientSecret"; String authorizationCode = "authorizationCode"; // Obtained from authorization server Response response = RestAssured .given() .contentType("application/x-www-form-urlencoded") .formParam("grant_type", "authorization_code") .formParam("code", authorizationCode) .formParam("redirect_uri", "https://your-redirect-uri.com") .formParam("client_id", clientId) .formParam("client_secret", clientSecret) .when() .post(tokenEndpoint); String accessToken = response.jsonPath().getString("access_token"); System.out.println("Access Token: " + accessToken); Once you have the access token, you can use it to make authenticated requests to your API endpoints Response apiResponse = RestAssured.given() .auth().oauth2(accessToken) .when() .get("https://api.example.com/protected-resource");	

Oauth 2.0

18 February 2023 13:00

OAuth is an authentication pattern used in REST APIs to manage access securely.

Why Authentication?
In a banking application example, after logging in, a single API call (e.g., "account overview") is made. This API internally calls multiple other APIs to gather details (e.g., balances, credit card info, profile details) and displays them. Proper authentication is needed not just for the initial user login but also for backend communication between different APIs.

Need for OAuth:
- Directly sharing customer credentials (username and password) with all internal APIs is insecure and impractical.
- OAuth introduces an Authorization Server to issue access tokens, which are then used to authenticate API calls securely.

OAuth Workflow:
1. Authorization Server:
  - Issues access tokens to internal services or applications.
  - Validates requests using grant types to ensure proper access control.
2. Grant Types(Way to access Token):
   - Client Credentials Grant: Uses a client ID and client secret to get an access token. Common in internal application interactions.
   - Password Grant: Uses username and password to obtain an access token. Used in some scenarios but less secure.
   - Authorization Code Grant: Another method for obtaining access tokens, often used in user-facing applications.

```
private String accessToken;
  @BeforeClass
  public void setUp() {
    RestAssured.baseURI = "https://rahulshettyacademy.com";
    Map<String, String> formParams = new HashMap<>();
    formParams.put("client_id", "692183103107-p0m7ent2hk7suguv4vq22hjcfhcr43pj.apps.googleusercontent.com");
    formParams.put("client_secret", "erZOWM9g3UtwNRj340YYaK_W");
    formParams.put("grant_type", "client_credentials");
    formParams.put("scope", "trust");

    String response = given()
        .formParams(formParams)
        .when()
        .post("oauthapi/oauth2/resourceOwner/token")
        .then()
        .statusCode(200)
        .extract()
        .response()
        .asString();

    JsonPath js = new JsonPath(response);
    accessToken = js.getString("access_token");
  }

  @Test
  public void getCourseDetails() {
    // Fetch course details using the access token
    String courseDetails = given()
        .queryParam("access_token", accessToken)
        .when()
        .get("oauthapi/getCourseDetails")
        .then()
        .statusCode(200) // Add status code assertion here for better test validation
        .extract()
        .response()
        .asString();
    System.out.println(courseDetails);
  }
```

--Content Type is: application/x-www-form-urlencoded

Format JSON

02 March 2023 18:11

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.ObjectWriter;
import com.fasterxml.jackson.databind.JsonNode;

// your JSON string
String jsonString = "{ \"name\": \"John\", \"age\": 30, \"city\": \"New York\" }";

// create an ObjectMapper object
ObjectMapper objectMapper = new ObjectMapper();

// read the JSON string into a JsonNode object
JsonNode jsonNode = objectMapper.readTree(jsonString);

// create an ObjectWriter with the pretty printer enabled
ObjectWriter objectWriter = objectMapper.writerWithDefaultPrettyPrinter();

// use the ObjectWriter to write the JsonNode as a pretty-printed JSON string
String prettyJsonString = objectWriter.writeValueAsString(jsonNode);

// print the pretty-printed JSON string
System.out.println(prettyJsonString);
```

Update JSON File

28 March 2023 07:56

BNY	"addlOrdData": [
	"AON",
	"NOTHELD"
	],
	
	// create a JSONObject object
	JSONObject jsonObject = new JSONObject(new String(Files.readAllBytes(Paths.get("src/test/resources/AddBook.json"))));
	
	// add a property to the JSONObject object
	jsonObject.put("propertyName", "propertyValue");
	
	// update the "addlOrddata" property with a JSONArray containing two strings
	JSONArray jsonArray = new JSONArray();
	jsonArray.put("aon");
	jsonArray.put("ng");
	jsonObject.put("addlOrddata", jsonArray);
	
	"solicited": false,
	
	// create a JSONObject object
	JSONObject jsonObject = new JSONObject();
	
	// set the initial value of the "solicited" property to false
	jsonObject.put("solicited", false);
	
	// update the value of the "solicited" property to true
	jsonObject.put("solicited", true);
	
	"commission": {
	"commType": "RTE",
	"commissionAmt": 2,
	"splitIP1Pct": null,
	"splitIP2Pct": null,
	"splitIP1": null,
	"splitIP2": null
	},
	
	// create a JSONObject object
	JSONObject jsonObject = new JSONObject();
	
	// update the value of the "commission" property with a nested object
	JSONObject commissionObject = new JSONObject();
	commissionObject.put("commtype", "rte");
	commissionObject.put("commvskue", 2);
	jsonObject.put("commission", commissionObject);

UpdateJsonWithJackson	@Test
	public void UpdateJsonWithJackson() throws JsonProcessingException {
	String json = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author \": \"John foer\" }";
	
	// Parse JSON into a Map
	ObjectMapper objectMapper = new ObjectMapper();
	Map jsonMap = objectMapper.readValue(json, Map.class);
	
	// Update the name field
	jsonMap.put("name", "Learn Selenium with Python");
	
	// Serialize back to JSON
	String updatedJson = objectMapper.writeValueAsString(jsonMap);
	System.out.println(updatedJson);
	}

UpdateJsonWithGson	@Test public void UpdateJsonWithGson () { String json = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author \": \"John foer\" }";

	// Parse JSON into a JsonObject
	JsonObject jsonObject = JsonParser.parseString(json).getAsJsonObject();
	
	// Update the name field
	jsonObject.addProperty("name", "Learn Selenium with Python");
	
	// Serialize back to JSON
	String updatedJson = jsonObject.toString();
	System.out.println(updatedJson);
	}

UpdateJsonWithOrgJson	@Test
	public void UpdateJsonWithOrgJson () {
	String json = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author \": \"John foer\" }";
	
	// Parse JSON into a JSONObject
	JSONObject jsonObject = new JSONObject(json);
	
	// Update the name field
	jsonObject.put("name", "Learn Selenium with Python");
	
	// Serialize back to JSON
	String updatedJson = jsonObject.toString();
	System.out.println(updatedJson);
	}

UpdateJsonWithJaywayJsonPath	import com.jayway.jsonpath.JsonPath;
	import net.minidev.json.JSONObject;
	@Test
	public void UpdateJsonWithJaywayJsonPath () throws JsonProcessingException {
	String json = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author \": \"John foer\" }";
	
	// Parse JSON
	JSONObject jsonObject = JsonPath.parse(json).json();
	
	// Update the name field
	jsonObject.put("name", "Learn Selenium with Python");
	
	// Serialize back to JSON
	String updatedJson = jsonObject.toJSONString();
	System.out.println(updatedJson);
	}

UpdateJsonWithJSONObject	import javax.json.Json;
	import javax.json.JsonObject;
	import javax.json.JsonReader;
	import java.io.StringReader;
	@Test
	public void UpdateJsonWithJaywayJsonPath (){
	String json = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author \": \"John foer\" }";
	
	// Parse JSON
	JsonReader reader = Json.createReader(new StringReader(json));
	JsonObject jsonObject = reader.readObject();
	
	// Update the name field
	JsonObject updatedJsonObject = Json.createObjectBuilder(jsonObject)
	.add("name", "Learn Selenium with Python")
	.build();
	
	// Serialize back to JSON
	System.out.println(updatedJsonObject.toString());
	}

UpdateJsonWithSImpleJSONObject	import org.json.simple.JSONObject;
	import org.json.simple.parser.JSONParser;
	@Test
	public void UpdateJsonWithJSONObject(){
	String json = "{ \"name\": \"Learn Appium Automation with Java\", \"isbn\": \"bcd1\", \"aisle\": \"2926\", \"author \": \"John foer\" }";
	
	// Parse JSON
	JSONParser parser = new JSONParser();
	JSONObject jsonObject = (JSONObject) parser.parse(json);

```
    // Update the name field
    jsonObject.put("name", "Learn Selenium with Python");

    // Serialize back to JSON
    System.out.println(jsonObject.toJSONString());
}
```

Miscellaneous

26 October 2024 18:26

Read Files	.body(new String(Files.readAllBytes(Paths.get("src/test/resources/AddPlace.json"))))
Object Mapper	ObjectMapper mapper = new ObjectMapper(); File file = new File("src/test/resources/complex.json"); ComplexJson.ComplexJsonData complexJsonData = mapper.readValue(file, ComplexJson.ComplexJsonData.class);
Assert and Extract	then().assertThat().statusCode(200).body("scope", equalTo("APP")) .header("server", "Apache").extract().response().asString()
Static Import	By using static imports, we can directly access these methods without prefixing them with the class name, which can makes the test code cleaner and more concise. For example, instead of writing RestAssured.given(), you can simply write given().
Method Chaining	In RestAssured, the method chaining on given(), when(), and then() is made possible through the use of the builder pattern. Each of these methods returns an object of the same class which implements Request Specification interface, this allows to chain them together. This improves readability of the code. Builder Pattern: The builder pattern is a design pattern in which an object is constructed step by step. Each step typically returns a modified builder object, allowing method chaining.
Sending Attachment	.multiPart("file", new File(ATTACHMENT_PATH)) In Postman this is sent in Body via Form Data
JSON Path	String postResponse = response.asString(); JsonPath js = new JsonPath(postResponse); String placeId = js.get("place_id");

List<Map<String, Object>> courses = jsonPath.getList("courses");	"courses": [
	{
	"title": "Selenium Python",
	"price": 50,
	"copies": 6,
	"sales": {
	"udemy": 20,
	"website": 5
	},
	"reviews": [
	{
	"channel": "udemy",
	"reviewcount": 20
	},
	{
	"channel": "website",
	"reviewcount": 5
	}
	]
	}

Map<String, Object> dashboard = jsonPath.getMap("dashboard");	"dashboard": {
	"purchaseAmount": 910,
	"website": "rahulshettyacademy.com"
	}

Response as Class	CourseDetails courseDetails = given().queryParam("access_token", accessToken).when().get("oauthapi/getCourseDetails").then().statusCode(200).extract().response().as(CourseDetails .class);
RestAssured.useRelaxedHTTPSValidation();	
URL Encoding	String response = given().urlEncodingEnabled(false).get("https://jira.atlassian.com:443/rest/api/2.0.alpha1/search?jql=project%20=%20BAM%20AND%20issuetype%20=%20Bug").asString(); ..
how to get response time from Response object in restassured	.time(lessThan(2000L)); // Milliseconds
What is the difference between URL and URI?	URL: Uniform Resource Locator, identifies a resource on the internet. URI: Uniform Resource Identifier, a more general term that includes URLs and other identifiers.
JSONObject	{ "page": 2, "per_page": 6, "total": 12, "total_pages": 2, "data": [

```
    {
      "id": 7,
      "email": "michael.lawson@reqres.in",
      "first_name": "Michael",
      "last_name": "Lawson",
      "avatar": "https://reqres.in/img/faces/7-image.jpg"
    },
    {
      "id": 8,
      "email": "lindsay.ferguson@reqres.in",
      "first_name": "Lindsay",
      "last_name": "Ferguson",
      "avatar": "https://reqres.in/img/faces/8-image.jpg"
    }
 ],
  "support": {
    "url": "https://contentcaddy.io?utm_source=reqres&utm_medium=json&utm_campaign=referral",
    "text": "Tired of writing endless social media content? Let Content Caddy generate it for you."
  }
}
```


```
JSONObject jsonObject=new JSONObject(response);
    System.out.println(jsonObject.get("page"));
    JSONArray data = jsonObject.getJSONArray("data");
    System.out.println("Data Array: " + data);
    for (int i = 0; i < data.length(); i++) {
      JSONObject record = data.getJSONObject(i);
      System.out.println("ID: " + record.getInt("id"));
    }

JSONObject support=jsonObject.getJSONObject("support");
    System.out.println(support.get("url"));
```

Response Validation

1. Validate Structure : Check that the response body follows the expected structure defined in the API documentation or specifications. This includes verifying the presence of required fields, nested objects, arrays, and data types.
2. Parse JSON/XML : If the response body is in JSON or XML format, parse it programmatically to extract relevant data for validation. Use libraries or tools appropriate for your programming language to parse and manipulate the response.
3. Verify Data Integrity: Ensure that the data returned in the response body is accurate and consistent with the input provided in the request. Compare the response data against the expected values or data retrieved from the database.
4. Handle Pagination: If the API supports pagination, analyze the response body to ensure that pagination parameters such as page numbers, page size, and total count are correctly returned. Test scenarios involving navigating through multiple pages of results.
5. Test Edge Cases: Evaluate how the API handles edge cases and boundary conditions in the response body. This includes scenarios such as empty responses, null values, maximum and minimum values, and unexpected input data.
6. Check Error Responses: Verify that error responses include meaningful error messages and appropriate HTTP status codes. Analyze the response body for error details such as error codes, descriptions, and troubleshooting instructions.
7. Evaluate Performance: Analyze the size and complexity of the response body to assess performance implications, especially for large datasets or complex object structures. Monitor response times and network traffic during performance testing.
8. Handle Localization: If the API supports localization or internationalization, analyze the response body to ensure that localized content is correctly returned based on the client's language preferences specified in the request.
9. Test Content Negotiation: If the API supports content negotiation, test scenarios where clients request specific media types (e.g., JSON, XML) in the Accept header. Analyze the response body to verify that the requested content type is returned.
10. Validate Hypermedia Controls: If the API follows the HATEOAS principle (Hypermedia as the Engine of Application State), analyze the response body to ensure that hypermedia links and controls are correctly included, allowing clients to navigate the API dynamically.

Security

Authentication	Validate token expiration, token revocation, replay attacks, and credential brute-forcing
Authorization	Check for privilege escalation, verify access control rules, and ensure that users cannot access data belonging to other users.
Data Validation and Input Handling	Analyze how APIs handle data validation and input handling. Assess how they respond to input formats, including invalid or unexpected data. Pay special attention to potential injection vulnerabilities
Error Handling and Exception Management:	Evaluate how APIs handle errors and exceptions. Test their response to different error conditions and ensure that sensitive information is not leaked in error messages

Test Cases	• Status Code Validation for Valid Requests • Authentication Handling with Invalid Credentials • Graceful Handling of Missing or Invalid Parameters • Input Data Validation with Malformed Data(extra data, invalid data, invalid data type, invalid json,empty json) • Timeout Handling under Load • Pagination Functionality Verification: • Schema Validation • Error Handling Capabilities for Meaningful Messages: Evaluate the API's error-handling capabilities by intentionally causing errors, such as invalid inputs or unexpected situations, and confirm that it consistently returns meaningful error messages for troubleshooting. • Response Time Measurement for Various Requests • Handling Large Payloads (File Uploads) • Validate sorting, pagination, filter param

JSON Schema Validation

23 November 2024 18:34

given the following schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "title": "Product set",
  "type": "array",
  "items": {
    "title": "Product",
    "type": "object",
    "properties": {
      "id": {
        "description": "The unique identifier for a product",
        "type": "number"
      },
      "name": {
        "type": "string"
      },
      "price": {
        "type": "number",
        "minimum": 0,
        "exclusiveMinimum": true
      },
      "tags": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "minItems": 1,
        "uniqueItems": true
      },
      "dimensions": {
        "type": "object",
        "properties": {
          "length": {"type": "number"},
          "width": {"type": "number"},
          "height": {"type": "number"}
        },
        "required": ["length", "width", "height"]
      },
      "warehouseLocation": {
        "description": "Coordinates of the warehouse with the product",
        "$ref": "http://json-schema.org/geo"
      }
    },
    "required": ["id", "name", "price"]
  }
}
```

```
import io.restassured.RestAssured;
import io.restassured.module.jsv.JsonSchemaValidator;
```

```
RestAssured.given()
    .baseUri("https://api.example.com")
    .when()
    .get("/users")
    .then()
    .assertThat()
    .body(matchesJsonSchema(new File("path/to/your/schema.json")));
```

JsonPath

25 December 2024 08:32

Read-only utility for extracting data from JSON responses.

```
import io.restassured.path.json.JsonPath;

public class JsonPathExample {
   public static void main(String[] args) {
      String jsonResponse = "{ \"store\": { \"book\": [ { \"category\": \"fiction\", \"author\": \"John\",
\"price\": 10 }, { \"category\": \"non-fiction\", \"author\": \"Jane\", \"price\": 15 } ], \"bicycle\":
{ \"color\": \"red\", \"price\": 19.95 } } }";

      // Parse JSON using JsonPath
      JsonPath jsonPath = new JsonPath(jsonResponse);

      // Extracting a simple value
      String bicycleColor = jsonPath.getString("store.bicycle.color");
      System.out.println("Bicycle Color: " + bicycleColor);

      // Extracting an array element
      String firstBookAuthor = jsonPath.getString("store.book[0].author");
      System.out.println("First Book Author: " + firstBookAuthor);

      // Extracting all elements in an array
      List<String> bookCategories = jsonPath.getList("store.book.category");
      System.out.println("Book Categories: " + bookCategories);

      // Filtering data with a condition
      int expensiveBookPrice = jsonPath.getInt("store.book.find { it.price > 12 }.price");
      System.out.println("Expensive Book Price: " + expensiveBookPrice);
   }
}
```

Common Methods in JsonPath
Basic Data Retrieval:

getString(path): Returns the value at the given path as a String.
getInt(path): Returns the value at the given path as an int.
getFloat(path): Returns the value at the given path as a float.
Extracting Arrays and Objects:

getList(path): Returns a list of values at the specified path.
getMap(path): Returns a map of key-value pairs.
Filtering and Searching:

find(): Searches for elements in a list that match a condition.
findAll(): Returns all elements in a list that match a condition.
Default Value:

getOrDefault(path, defaultValue): Returns the value at the specified path, or the default value if the path does not exist.

JackSon

25 December 2024 09:36

Convert Java objects to JSON (serialization) and vice versa (deserialization).
Offers annotations like @JsonProperty, @JsonIgnore, and @JsonInclude for fine-grained control over serialization/deserialization.
Highly optimized for performance in both serialization and deserialization.

Useful Classes

ObjectMapper

Central class for reading and writing JSON.
Supports both data binding and tree model access.
JsonNode

Represents a node in a JSON tree.
Provides methods for traversing and manipulating JSON trees.

ObjectMapper

Method	Description
writeValueAsString(Object obj)	Serialize a Java object into a JSON string.
writeValue(File file, Object obj)	Serialize a Java object into a JSON file.
readValue(String json, Class<T> clazz)	Deserialize a JSON string into a Java object.
readTree(String json)	Parse a JSON string into a JsonNode tree.
convertValue(Object fromValue, Class<T> toValueType)	Convert one Java object to another using JSON as an intermediary.
writerWithDefaultPrettyPrinter()	Create a ObjectWriter that outputs pretty-printed JSON.
setSerializationInclusion(JsonInclude.Include inclusion)	Configure inclusion rules for serialization (e.g., ignore null values).

JsonNode Methods

Method	Description
get(String fieldName)	Retrieve a child node by field name.
asText()	Convert the value of the node to a String.
isArray() / isObject()	Check if the node is an array or an object.
size()	Get the size of an array or object node.
path(String fieldName)	Navigate to a child node; returns a missing node if the field does not exist.

Annotations	
Annotation	**Description**
@JsonProperty	Map a JSON property to a specific Java field.
@JsonIgnore	Ignore a field during serialization or deserialization.
@JsonInclude	Include/exclude fields based on certain rules (e.g., non-null values).
@JsonFormat	Specify the format for Date and other custom types.
@JsonDeserialize	Specify a custom deserializer class for a field.
@JsonSerialize	Specify a custom serializer class for a field.

3. Tree Model Manipulation
java
Copy code

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.node.ObjectNode;

public class JacksonTreeModelExample {
    public static void main(String[] args) throws Exception {
        String json = "{ \"name\": \"Learn Jackson\", \"isbn\": \"abc1\", \"aisle\": \"1234\", \"author\": \"John Doe\" }";

        ObjectMapper objectMapper = new ObjectMapper();

        // Parse JSON into a tree
        ObjectNode rootNode = (ObjectNode) objectMapper.readTree(json);

        // Update fields
        rootNode.put("name", "Learn Gson");
        rootNode.put("newField", "Added via Jackson");

        // Serialize back to JSON string
        String updatedJson = objectMapper.writeValueAsString(rootNode);
        System.out.println(updatedJson);
    }
}
```